Multimedia Computing

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University.

Email: minhaz.ahmed@gmail.com

Contents

- Feature detection
- Evaluating the results
- ROC
- Different face detection methods
- Object detection
- Face detection using haar features
- Cascade Classifier

Feature detection

- Feature detection and matching are an essential component of many computer vision applications.

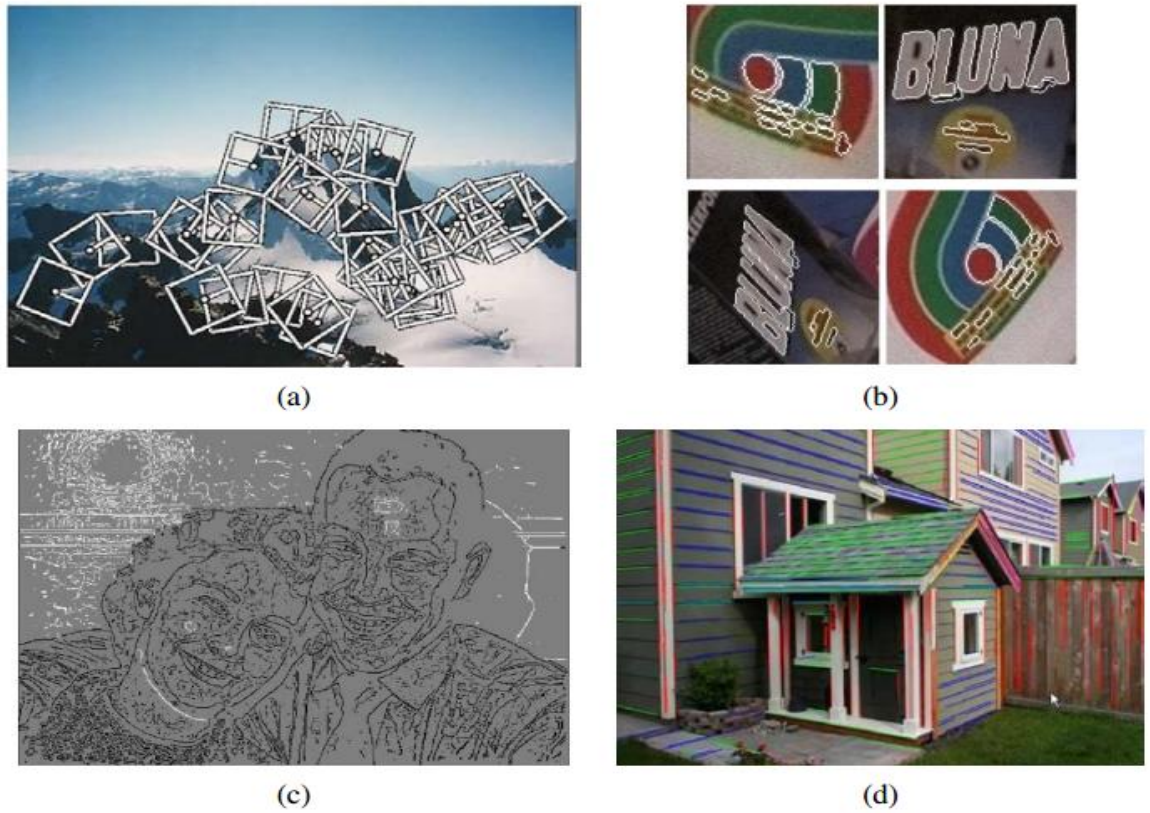


Figure 4.1 A variety of feature detectors and descriptors can be used to analyze, describe and match images:
(a) point-like interest operators (Brown, Szeliski, and Winder 2005) IEEE;
(b) region-like interest operators (Matas, Chum, Urban et al. 2004 (c) edges (Elder and Goldberg 2001)
(d) straight lines (Sinha, Steedly, Szeliski et al. 2008)

Face detection

Haar Cascade Method

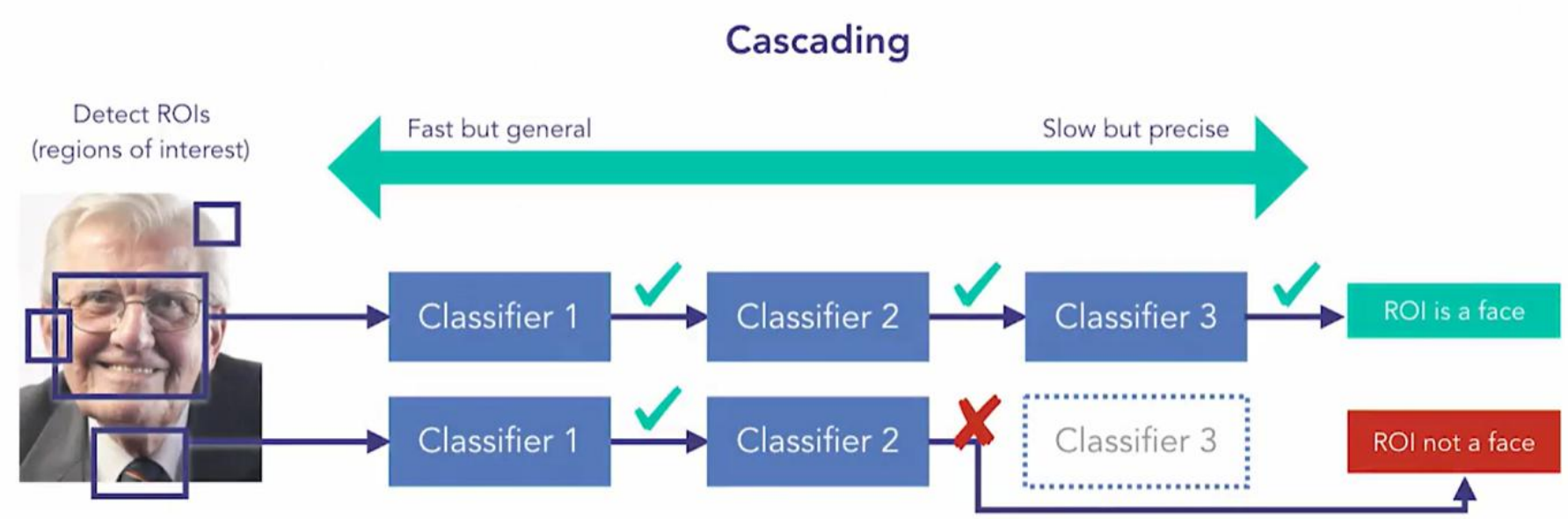
A form of features-based machine learning

Uses pretrained images of labeled positives and negatives

Runs through thousands of classifiers in a cascaded manner

Use case: detect faces in the image and draw bounding boxes

Face detection



Face detection

Face detection

- **Haar Cascade Method**

  A form of features-based machine learning

  Uses pretrained images of labeled positives and negatives

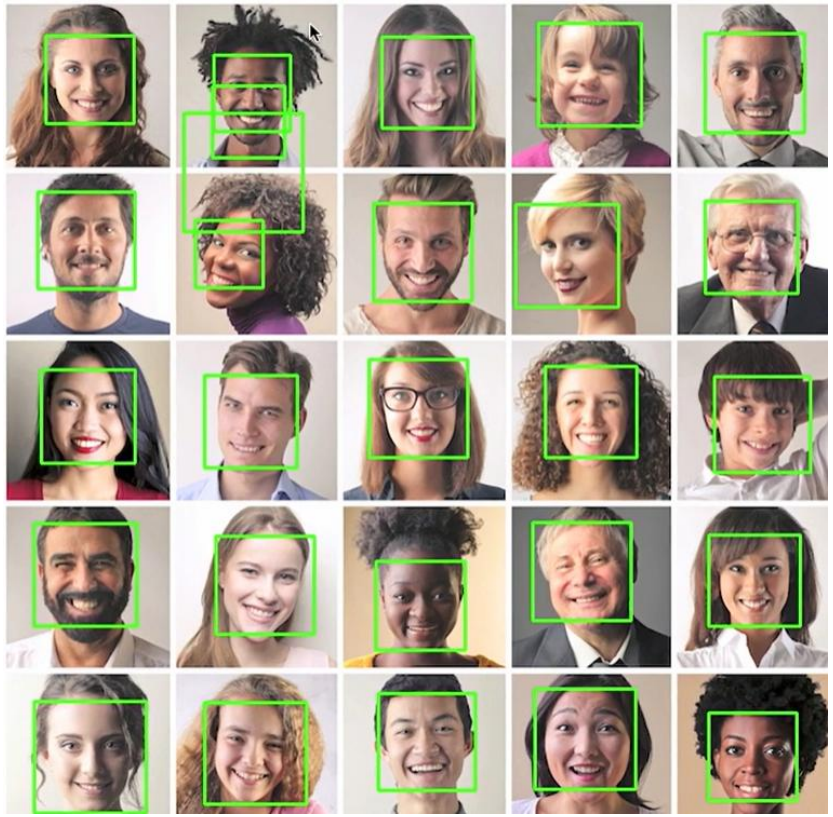
  Runs through thousands of classifiers in a cascaded manner

  Use case: detect faces in the image and draw bounding boxes

Face detection

```
import numpy as np
import cv2
#from google.colab.patches import cv2_imshow

img = cv2.imread("faces.jpeg",1)
gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
path = "haarcascade_frontalface_default.xml"

face_cascade = cv2.CascadeClassifier(path)

faces = face_cascade.detectMultiScale(gray, scaleFactor=1.10, minNeighbors=5,
minSize=(40,40))
print(len(faces))

for (x, y, w, h) in faces:
  cv2.rectangle(img, (x,y), (x+w,y+h), (0,255,0), 2)
cv2.imshow("Image",img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Face detection

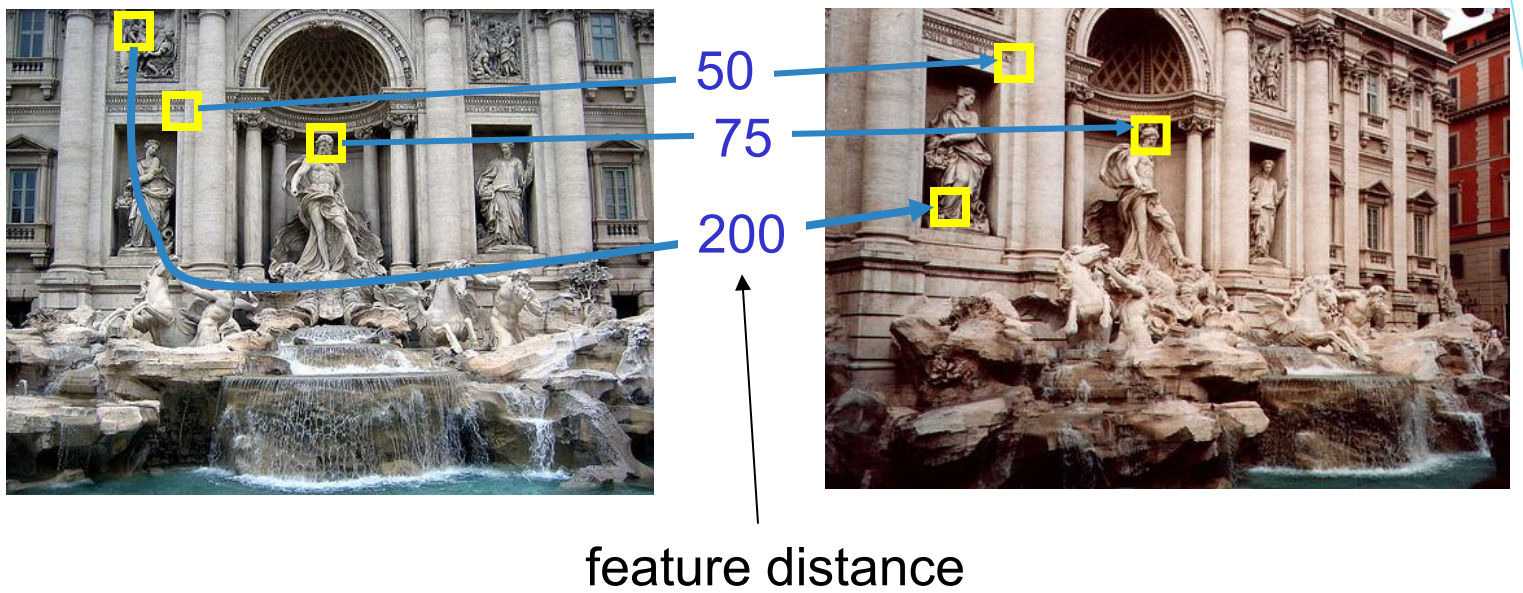


Feature detection

- The first kind of feature that you may notice are specific locations in the images, such as mountain peaks, building corners, doorways, or interestingly shaped patches of snow.
- These kinds of localized feature are often called keypoint features or interest points
- Another class of important features are edges, e.g., the profile of mountains against the sky. These kinds of features can be matched based on their orientation and local appearance (edge profiles) and can also be good indicators of object boundaries and occlusion events in image sequences.

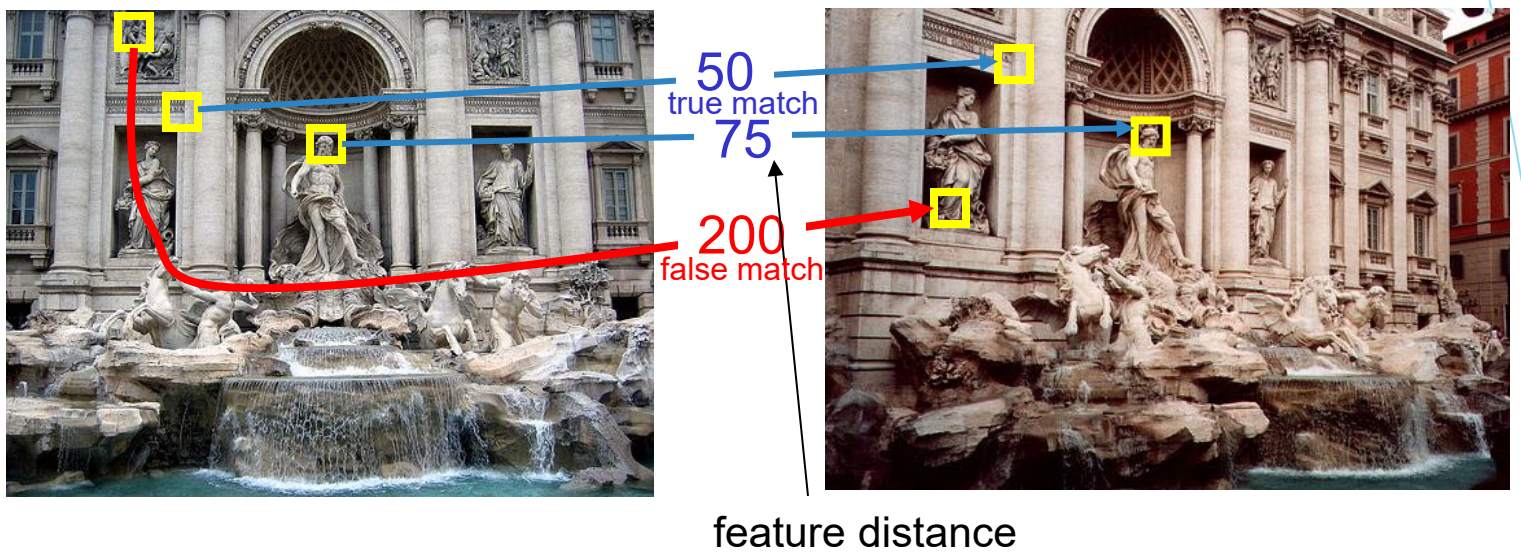
Evaluating the results

How can we measure the performance of a feature matcher?



Slide: S. Seitz

True/false positives



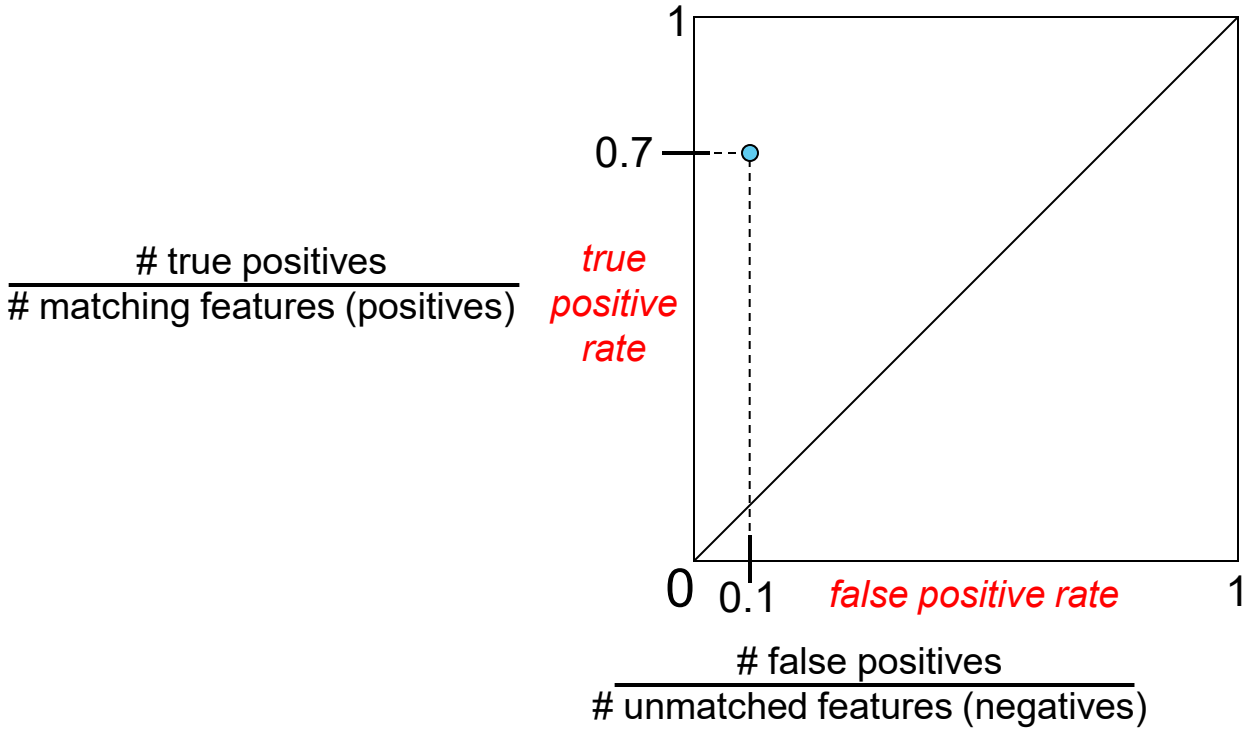
The distance threshold affects performance

- True positives = # of detected matches that are correct
  - Suppose we want to maximize these—how to choose threshold?
- False positives = # of detected matches that are incorrect
  - Suppose we want to minimize these—how to choose threshold?

Slide: S. Seitz

Evaluating the results

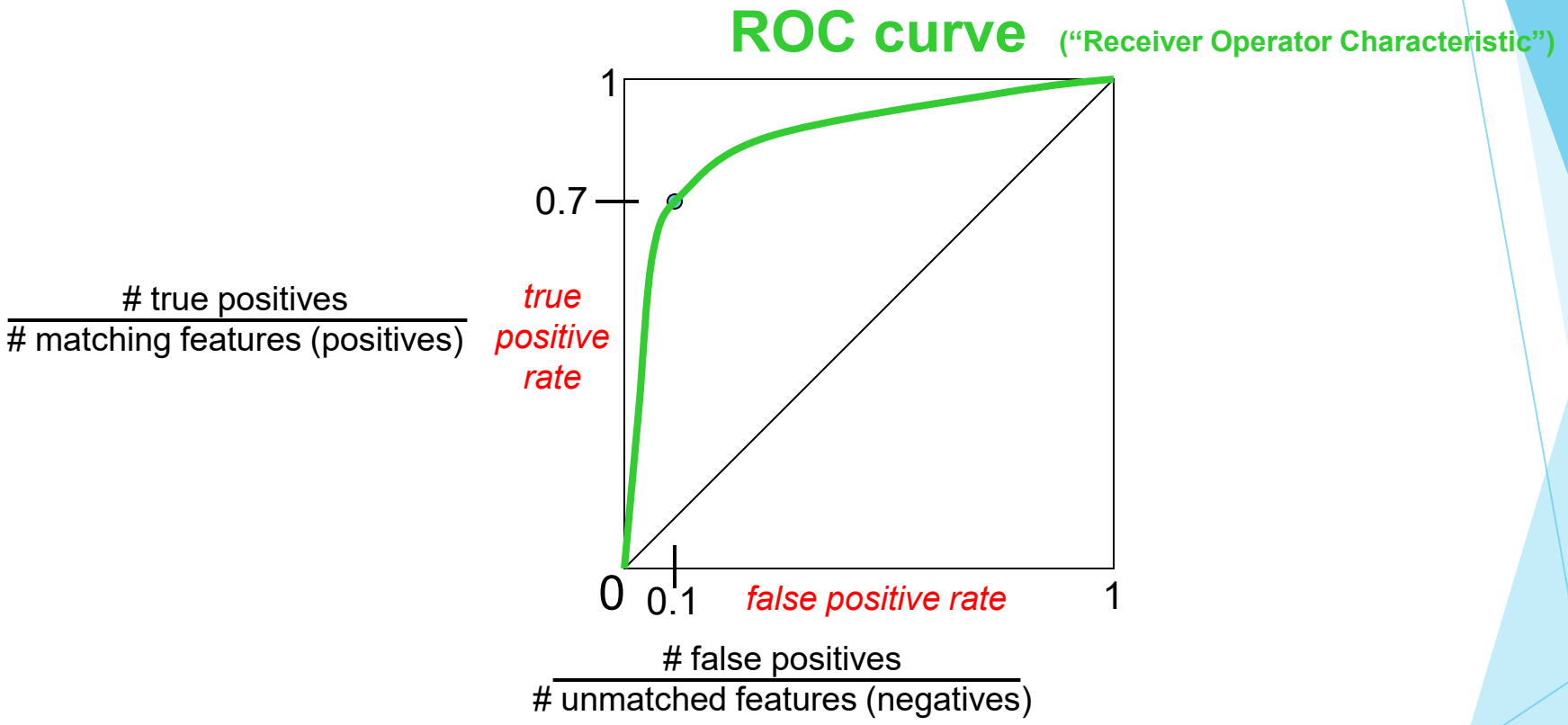
How can we measure the performance of a feature matcher?



Slide: S. Seitz

Evaluating the results

How can we measure the performance of a feature matcher?



ROC Curves

- Generated by counting # current/incorrect matches, for different thresholds
- Want to maximize area under the curve (AUC)
- Useful for comparing different feature matching methods
- For more info: http://en.wikipedia.org/wiki/Receiver_operating_characteristic

Evaluating the results (Example)

	True matches	True non-matches		
Predicted matches	TP = 18	FP = 4	P' = 22	PPV = 0.82
Predicted non-matches	FN = 2	TN = 76	N' = 78	
	P = 20	N = 80	Total = 100	
	TPR = 0.90	FPR = 0.05		ACC = 0.94

Table 4.1 The number of matches correctly and incorrectly estimated by a feature matching algorithm, showing the number of true positives (TP), false positives (FP), false negatives (FN) and true negatives (TN). The columns sum up to the actual number of positives (P) and negatives (N), while the rows sum up to the predicted number of positives (P') and negatives (N'). The formulas for the true positive rate (TPR), the false positive rate (FPR), the positive predictive value (PPV), and the accuracy (ACC) are given in the text.

first counting the number of true and false matches and match failures, using the following definitions (Fawcett 2006):

Evaluating the results (Example)

- TP: true positives, i.e., number of correct matches;
- FN: false negatives, matches that were not correctly detected;
- FP: false positives, proposed matches that are incorrect;
- TN: true negatives, non-matches that were correctly rejected.

Table 4.1 shows a sample *confusion matrix* (contingency table) containing such numbers.

We can convert these numbers into *unit rates* by defining the following quantities (Fawcett 2006):

- true positive rate (TPR),

$$\mathrm{TPR} = \frac{\mathrm{TP}}{\mathrm{TP+FN}} = \frac{\mathrm{TP}}{\mathrm{P}}; \tag{4.14}$$

- false positive rate (FPR),

$$\mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{FP+TN}} = \frac{\mathrm{FP}}{\mathrm{N}}; \tag{4.15}$$

- positive predictive value (PPV),

$$\mathrm{PPV} = \frac{\mathrm{TP}}{\mathrm{TP+FP}} = \frac{\mathrm{TP}}{\mathrm{P'}}; \tag{4.16}$$

- accuracy (ACC),

$$\mathrm{ACC} = \frac{\mathrm{TP+TN}}{\mathrm{P+N}}. \tag{4.17}$$

So what does object recognition involve?

Classification: does this contain people?

Detection: where are there people (if any)?

Detection and Recognition

- Analyzing a scene and recognizing all of the constituent objects remains the most challenging.

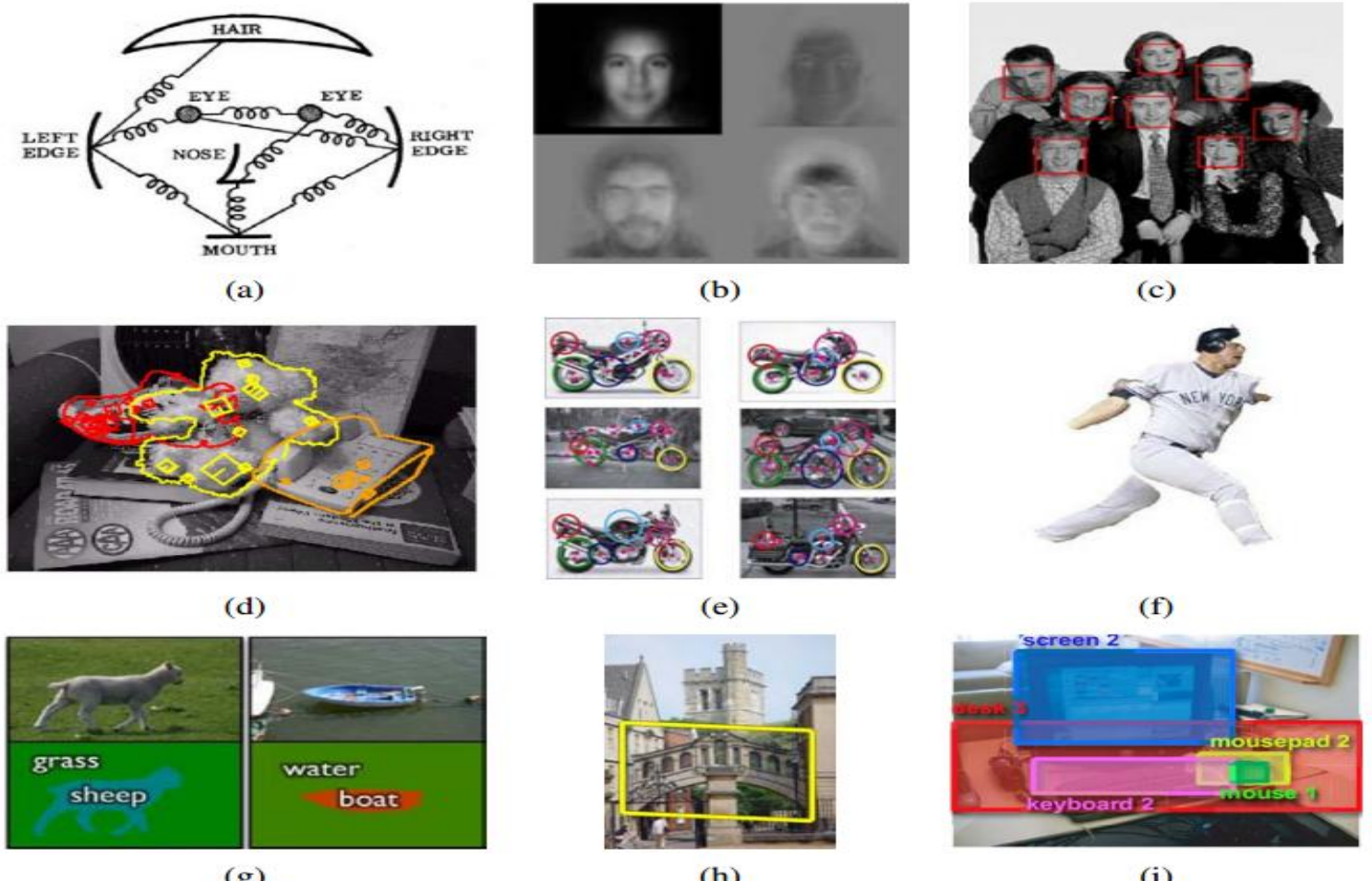


Figure 14.1 Recognition: face recognition with (a) pictorial structures (Fischler and Elschlager 1973 and (b) eigenfaces (Turk and Pentland 1991b); (c) realtime face detection (Viola and Jones 2004) (d) instance (known object) recognition (Lowe 1999) (e) feature-based recognition (Fergus Zisserman 2007); (f) region-based recognition (Mori, Ren, Efros et al. 2004) (g) simultaneous recognition and segmentation (Shotton, Winn, Rother et al. 2009) Springer; (h) location recognition (Philbin, Chum, 2007 IEEE; (i) using context (Russell, Torralba, Liu et al. 2007)

Face detection

- If we are given an image to analyze, such as the group portrait in Figure 14.2, we could try to apply a recognition algorithm to every possible sub-window in this image.

Figure 14.2 Face detection results produced by Rowley, Baluja, and Kanade (1998a) © 1998 IEEE. Can you find the one false positive (a box around a non-face) among the 57 true positive results?

Face detection

- In principle, we could apply a face recognition algorithm at every pixel and scale (Moghaddam and Pentland 1997) but such a process would be too slow in practice.

- Template-based approaches, such as active appearance models (AAMs) (Section 14.2.2),can deal with a wide range of pose and expression variability.

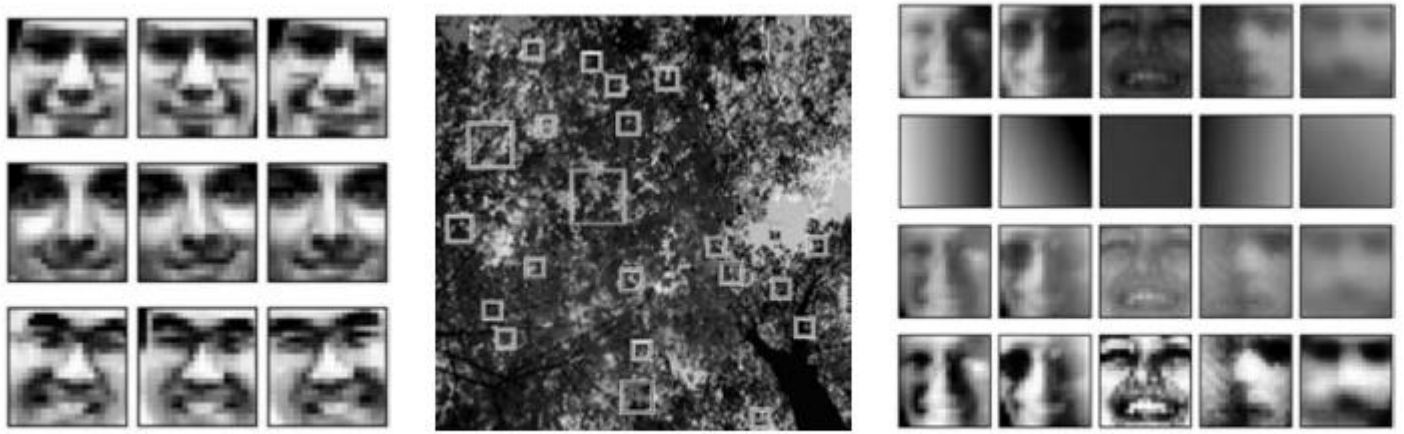


Figure 14.3 Pre-processing stages for face detector training (Rowley (a) artificially mirroring, rotating, scaling, and translating training images for greater variability; (b) using images without faces (looking up at a tree) to generate non-face examples; (c) pre-processing the patches by subtracting a best fit linear function and histogram equalizing

Face detection

- after an initial set of training images has been collected, some optional pre-processing can be performed, such as subtracting an average gradient .

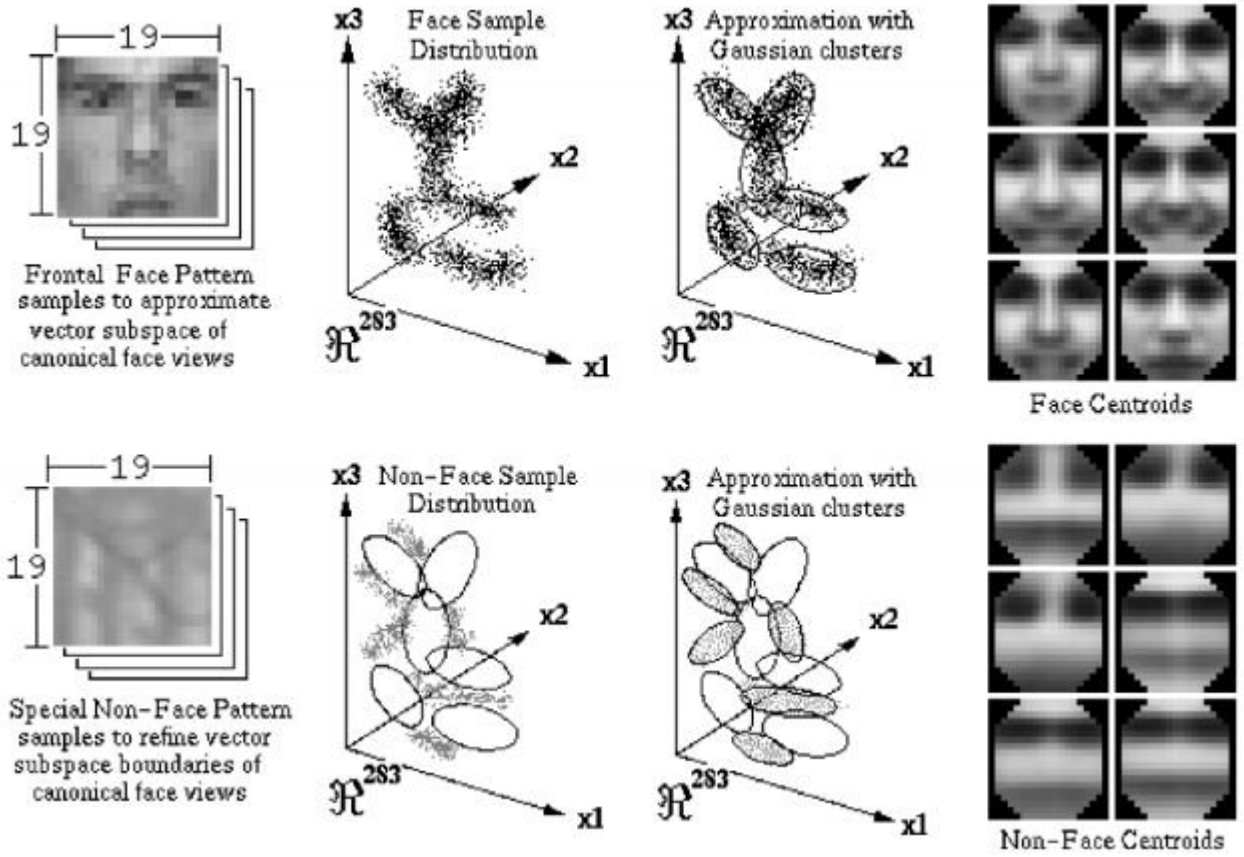


Figure 14.4 Learning a mixture of Gaussians model for face detection (Sung and Poggio 1998) The face and non-face images (192-long vectors) are first clustered into six separate clusters (each) using k-means and then analyzed using PCA. The cluster centers are shown in the right-hand columns.

Face detection

- Boosting. Face detectors by Viola and Jones (2004) is probably the best known and most widely used.
- Their technique was the first to introduce the concept of boosting to the computer vision community, which involves training a series of increasingly discriminating simple classifiers and then blending their outputs.

Face detection

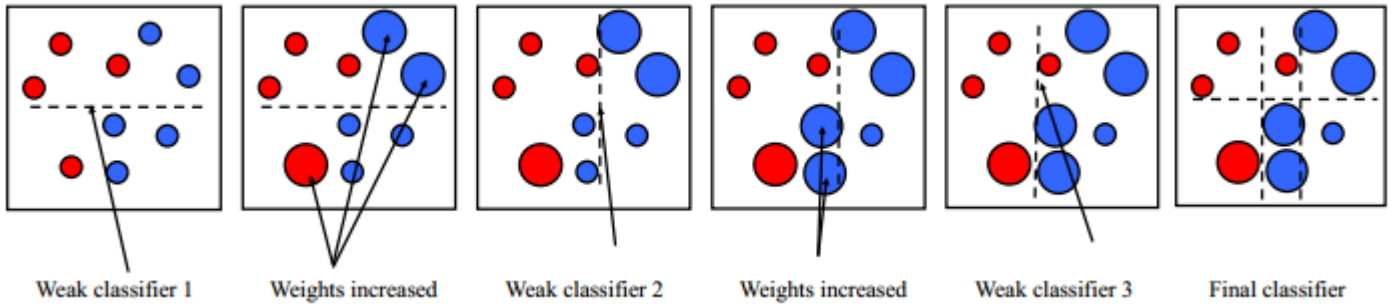


Figure 14.7 Schematic illustration of boosting, courtesy of Svetlana Lazebnik, after original illustrations from Paul Viola and David Lowe. After each weak classifier (decision stump or hyperplane) is selected, data points that are erroneously classified have their weights increased. The final classifier is a linear combination of the simple weak classifiers.

Face detection

- The boosting algorithm selects one of the input vector components as the best one to threshold.

- Essentially, for the cost of an O(N) pre-computation phase (where N is the number of pixels in the image), subsequent differences of rectangles can be computed in 4r additions or subtractions, where $r \in \{2, 3, 4\}$ is the number of rectangles in the feature.
- The key to the success of boosting is the method for incrementally selecting the weak learners and for re-weighting the training examples after each stage

using the stage-wise average classification error to determine the final weightings αj among the weak classifiers, as described in Algorithm 14.1.

1. Input the positive and negative training examples along with their labels $\{(\boldsymbol{x}_i, y_i)\}$, where $y_i = 1$ for positive (face) examples and $y_i = -1$ for negative examples.

2. Initialize all the weights to $w_{i,1} \leftarrow \frac{1}{N}$, where N is the number of training examples. (Viola and Jones (2004) use a separate N_1 and N_2 for positive and negative examples.)

3. For each training stage $j = 1 \ldots M$:

   (a) Renormalize the weights so that they sum up to 1 (divide them by their sum).

   (b) Select the best classifier $h_j(\boldsymbol{x}; f_j, \theta_j, s_j)$ by finding the one that minimizes the weighted classification error

$$e_j = \sum_{i=0}^{N-1} w_{i,j} e_{i,j}, \tag{14.3}$$

$$e_{i,j} = 1 - \delta(y_i, h_j(\boldsymbol{x}_i; f_j, \theta_j, s_j)). \tag{14.4}$$

   For any given f_j function, the optimal values of (θ_j, s_j) can be found in linear time using a variant of weighted median computation (Exercise 14.2).

   (c) Compute the modified error rate β_j and classifier weight α_j,

$$\beta_j = \frac{e_j}{1 - e_j} \quad \text{and} \quad \alpha_j = -\log \beta_j. \tag{14.5}$$

   (d) Update the weights according to the classification errors $e_{i,j}$

$$w_{i,j+1} \leftarrow w_{i,j} \beta_j^{1-e_{i,j}}, \tag{14.6}$$

   i.e., downweight the training samples that were correctly classified in proportion to the overall classification error.

4. Set the final classifier to

$$h(\boldsymbol{x}) = \operatorname{sign}\left[\sum_{j=0}^{m-1} \alpha_j h_j(\boldsymbol{x})\right]. \tag{14.7}$$

Algorithm 14.1 The AdaBoost training algorithm, adapted from Hastie, Tibshirani, and Friedman (2001), Viola and Jones (2004)

Timeline of recognition

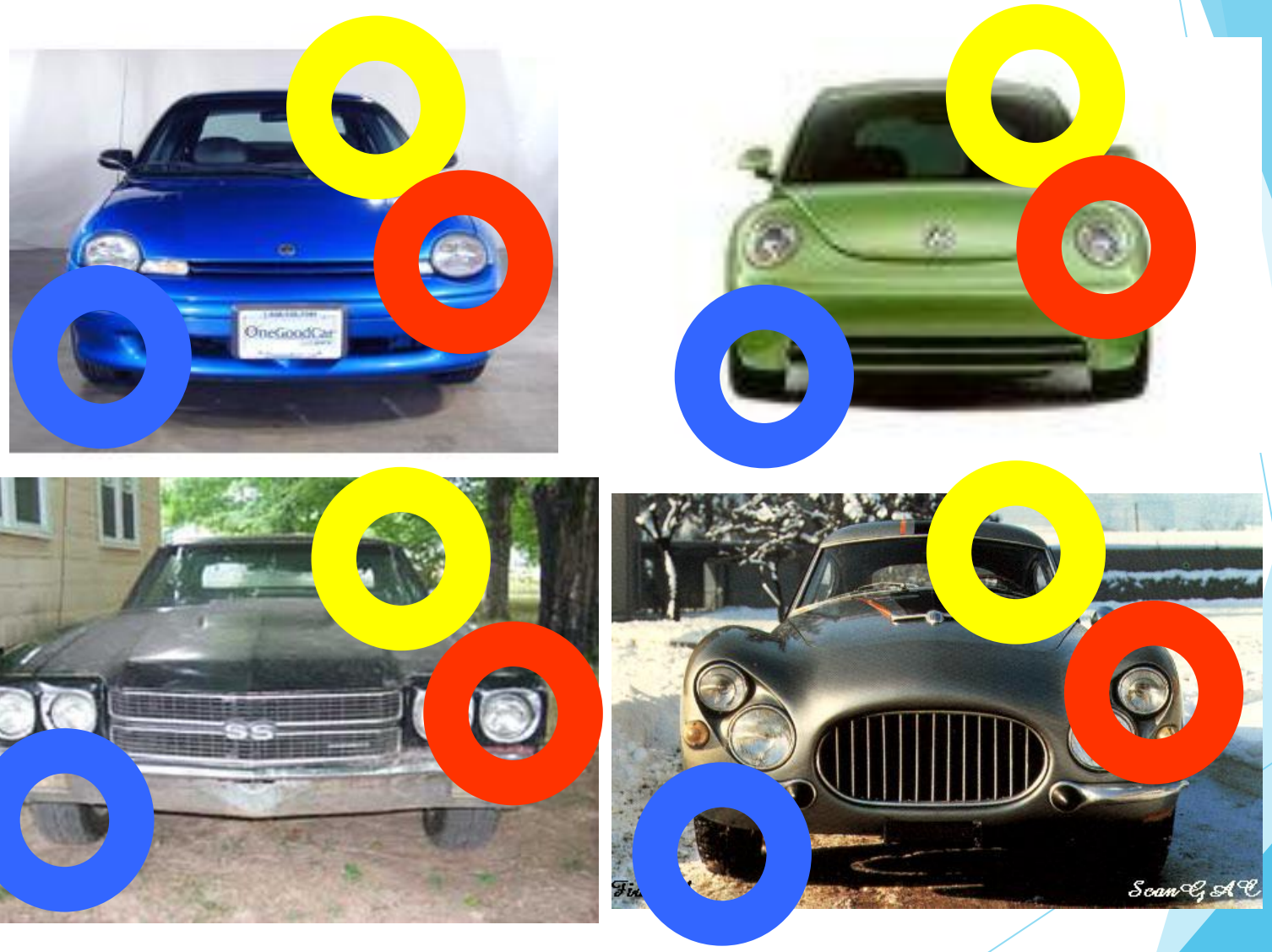
- 1965-late 1980s: alignment, geometric primitives
- Early 1990s: invariants, appearance-based methods
- Mid-late 1990s: sliding window approaches
- Late 1990s: feature-based methods
- Early 2000s – present : parts-and-shape models
- 2003 – 2010: bags of features, HOG
- Present trends: combination of local and global methods, modeling context, integrating recognition and segmentation
- 2012- Deep learning

Pedestrian detection

- An example of a well-known pedestrian detector is the algorithm developed by Dalal and Triggs (2005), who use a set of overlapping histogram of oriented gradients (HOG)

Figure 14.8 Pedestrian detection using histograms of oriented gradients (Dalal and Triggs 2005) (a) the average gradient image over the training examples; (b) each “pixel” shows the maximum positive SVM weight in the block centered on the pixel; (c) likewise, for the negative SVM weights; (d) a test image; (e) the computed R-HOG (rectangular histogram of gradients) descriptor; (f) the R-HOG descriptor weighted by the positive SVM weights; (g) the R-HOG descriptor weighted by the negative SVM weights.

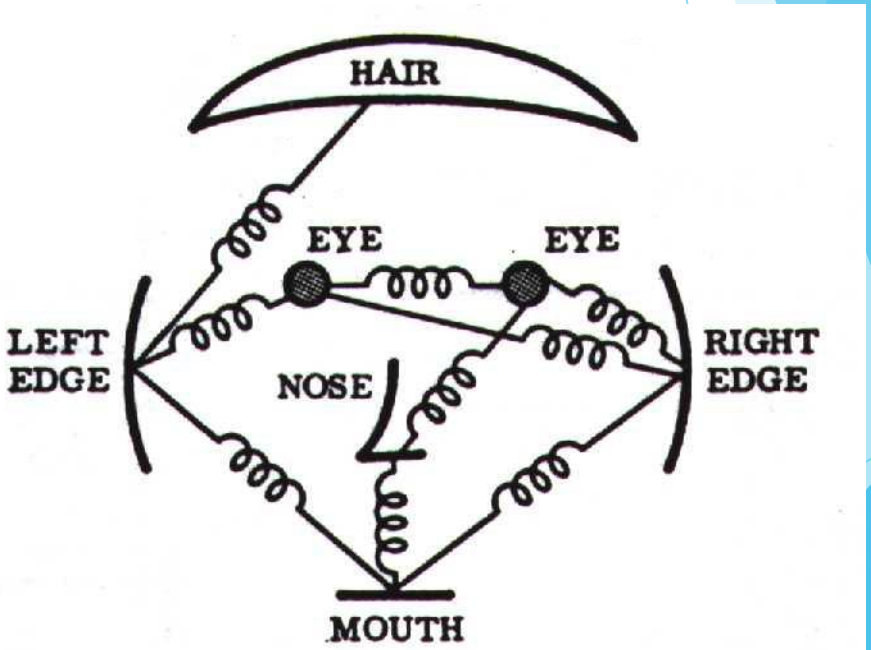
Representing categories: Parts and Structure



Weber, Welling & Perona (2000), Fergus, Perona & Zisserman (2003)

Representation

- Object as set of parts
  - Generative representation
- Model:
  - Relative locations between parts
  - Appearance of part
- Issues:
  - How to model location
  - How to represent appearance
  - Sparse or dense (pixels or regions)
  - How to handle occlusion/clutter



Bag-of-features models

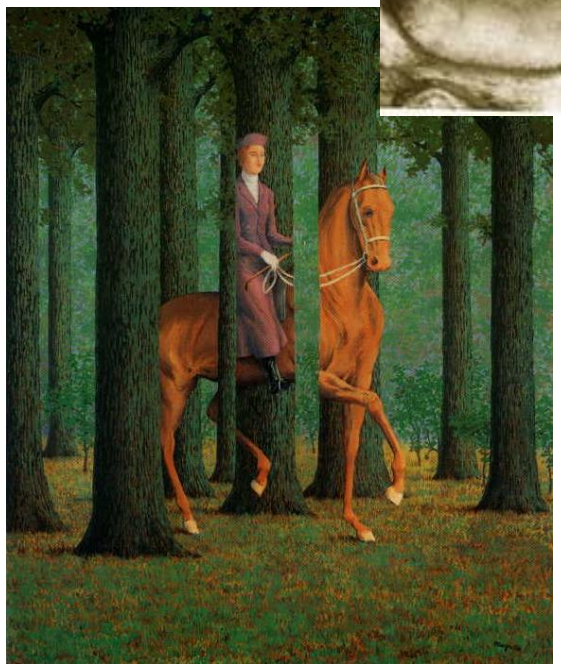
Object → Bag of 'words'

Three main issues

- Representation
  - How to represent an object category
- Learning
  - How to form the classifier, given training data
- Recognition
  - How the classifier is to be used on novel data

Representation

- Generative / discriminative / hybrid
- Appearance only or location and appearance
- Invariances
  - View point
  - Illumination
  - Occlusion
  - Scale
  - Deformation
  - Clutter
  - etc.



Learning

- Unclear how to model categories, so we learn what distinguishes them rather than manually specify the difference -- hence current interest in machine learning

Applications today

- Reading license plates, zip codes, checks
- Fingerprint recognition
- Face detection

Face detection using *haar-like features*

- Haar cascades, it is talking about cascade classifiers based on Haar features. To understand what this means, we need to take a step back and understand why we need this in the first place.
- Back in 2001, Paul Viola and Michael Jones came up with a very effective object detection method in their seminal paper. It has become one of the major landmarks in the field of machine learning

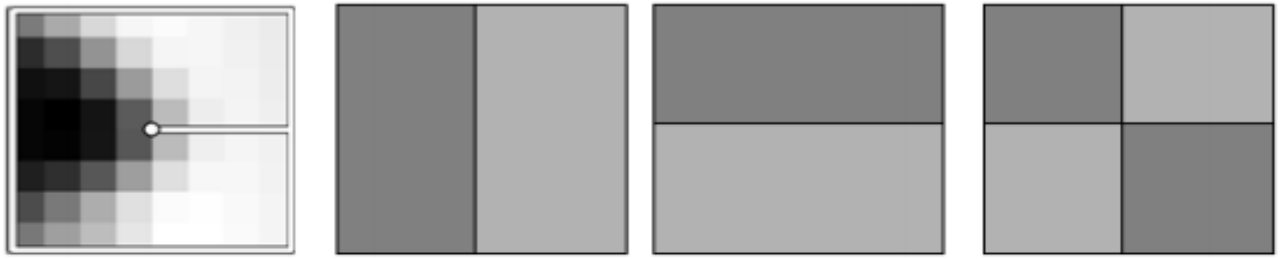


Figure 4.26 The three Haar wavelet coefficients used for hashing the MOPS descriptor devised by Brown, Szeliski, and Winder (2005) are computed by summing each 8×8 normalized patch over the light and dark gray regions and taking their difference.

Face detection using *haar-like features*

- First, a classifier (namely a *cascade of boosted classifiers working with haar-like features*) is trained with a few hundred sample views of a particular object (i.e., a face or a car), called positive examples, that are scaled to the same size (say, 20x20), and negative examples - arbitrary images of the same size.

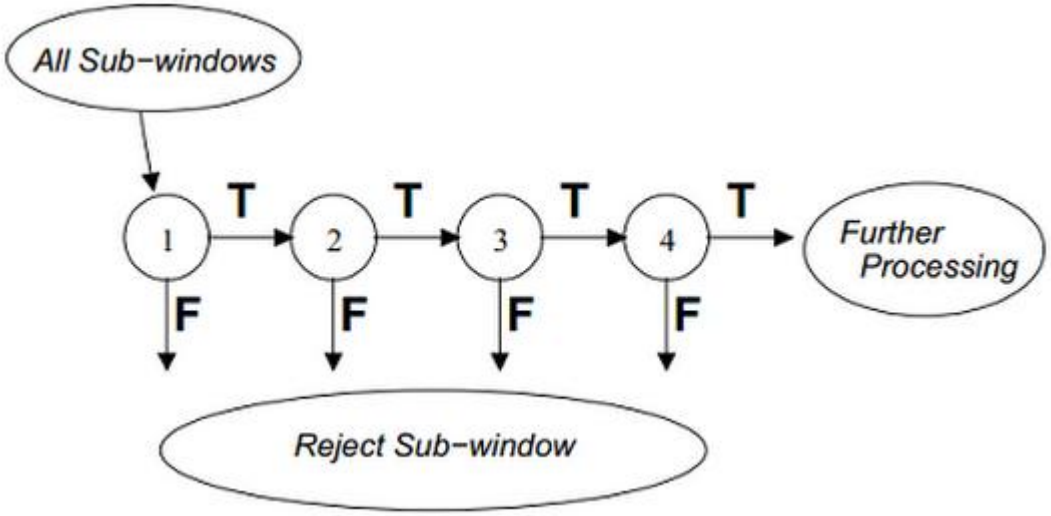


Fig. Cascade classifier

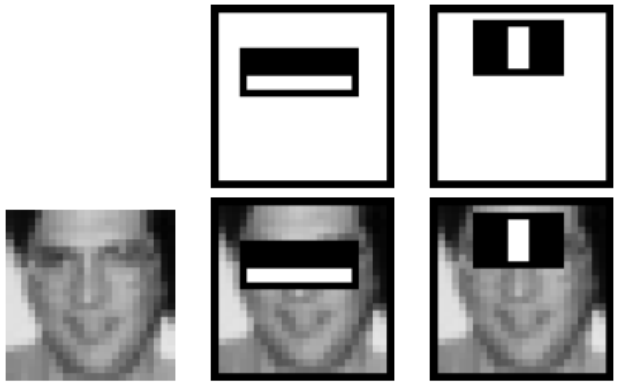
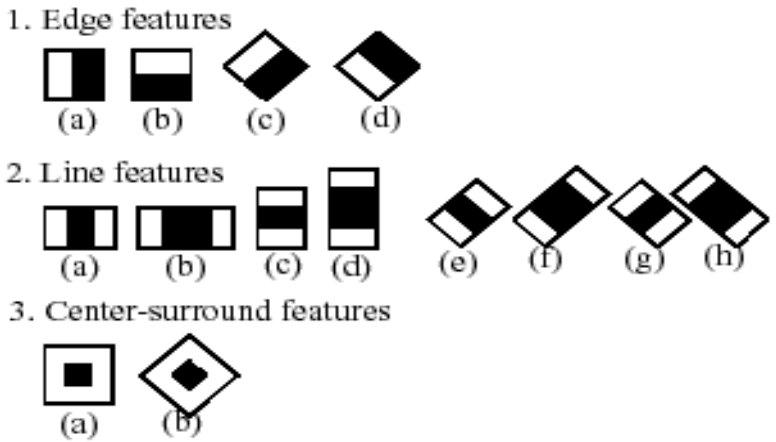
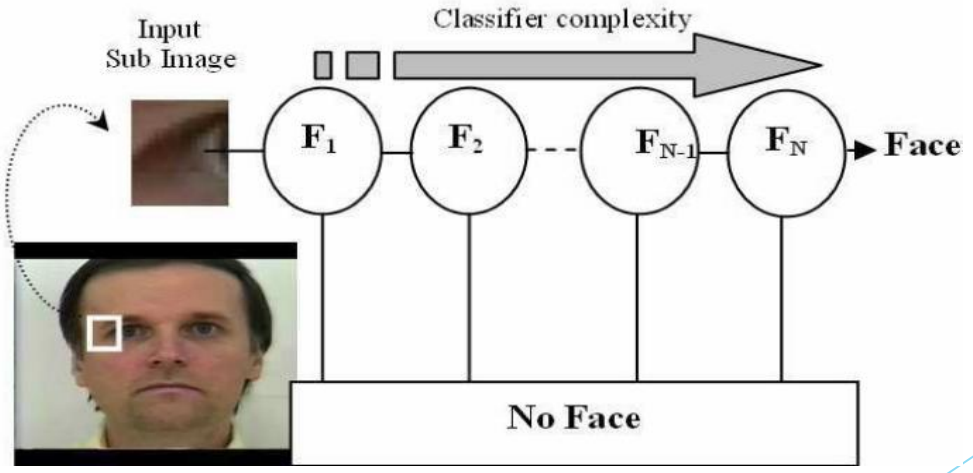


Fig. Haar like feature

Haar Feature-based Cascade Classifier for Object Detection

After a classifier is trained, it can be applied to a region of interest (of the same size as used during the training) in an input image. The classifier outputs a "1" if the region is likely to show the object (i.e., face/car), and "0" otherwise. To search for the object in the whole image one can move the search window across the image and check every location using the classifier

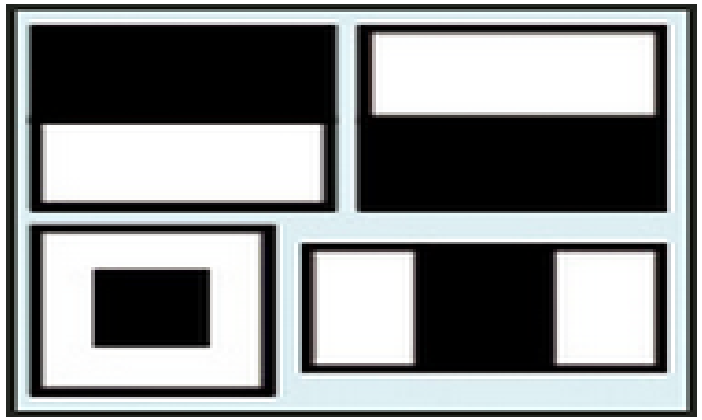


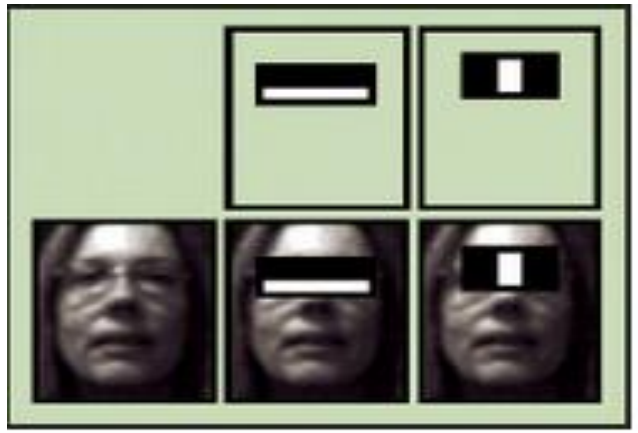


How face detector work

- Simple rectangular features, called Haar features
- An Integral Image for rapid feature detection
- The AdaBoost machine-learning method
- A cascaded classifier to combine many features efficiently

The features that Viola and Jones used are based on Haar wavelets. Haar wavelets are single wavelength square waves (one high interval and one low interval). In two dimensions, a square wave is a pair of adjacent rectangles - one light and one dark.





Use Integral Image

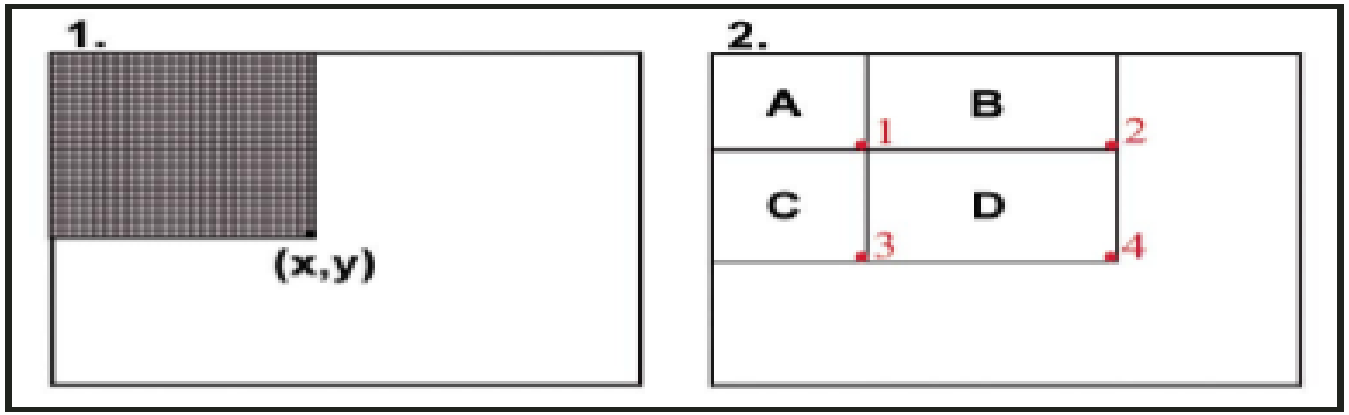


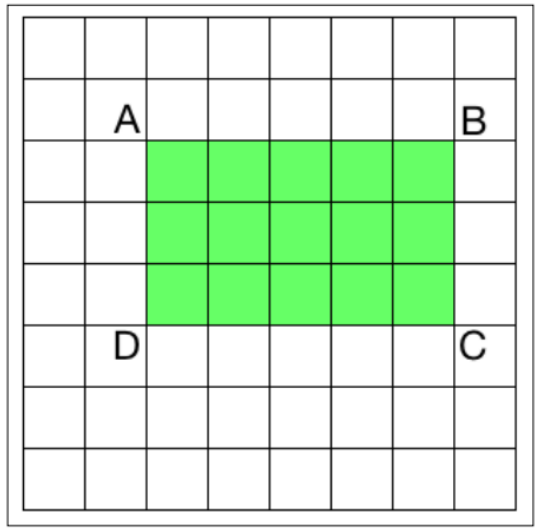
Figure 2. (Click for larger view.) The Integral Image trick.
a.After integrating, the pixel at (x,y) contains the sum of all pixel values in the shaded rectangle.
b.The sum of pixel values in rectangle D is
(x4, y4) - (x2, y2) - (x3, y3) + (x1, y1).

The presence of a Haar feature is determined by subtracting the average dark-region pixel value from the average light-region pixel value. If the difference is above a threshold (set during learning), that feature is said to be present.

To determine the presence or absence of hundreds of Haar features at every image location and at several scales efficiently, Viola and Jones used a technique called an Integral Image.

Integral Image

While compute Haar features, we will have to compute the summations of many different rectangular regions within the image. If we want to effectively build the feature set, we need to compute these summations at multiple scales. This is a very expensive process! If we want to build a real time system, we cannot spend so many cycles in computing these sums. So we use something called integral images.

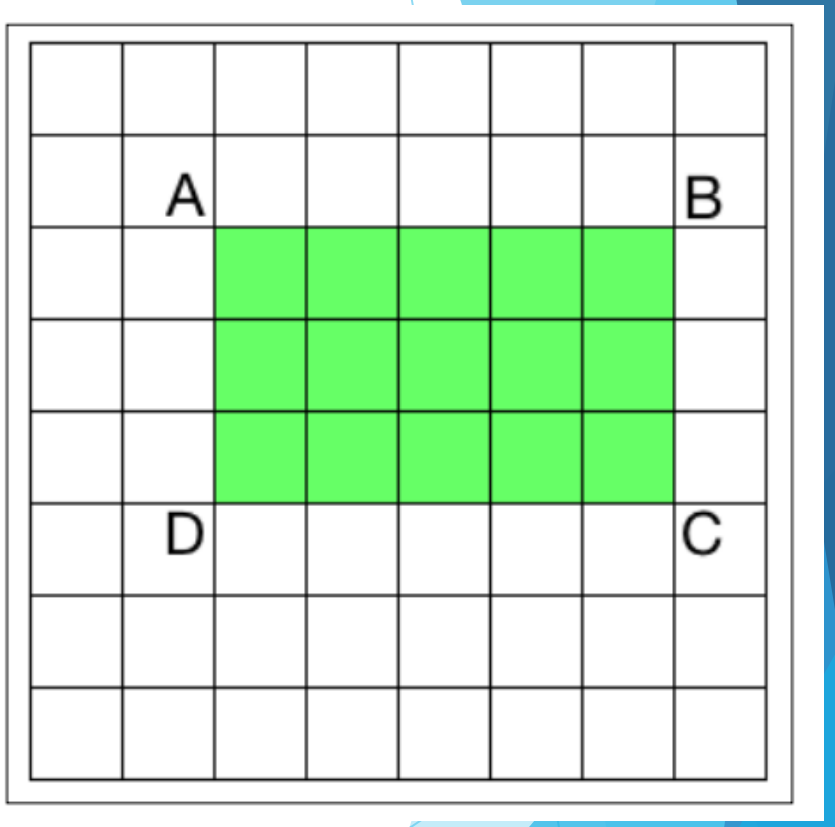


Integral Image

To compute the sum of any rectangle in the image, we don't need to go through all the elements in that rectangular area.

Let's say AP indicates the sum of all the elements in the rectangle formed by the top left point and the point P in the image as the two diagonally opposite corners. So now, if we want to compute the area of the rectangle ABCD, we can use the following formula:

Area of the rectangle ABCD = AC – (AB + AD - AA)



Integral Image

Input Image:

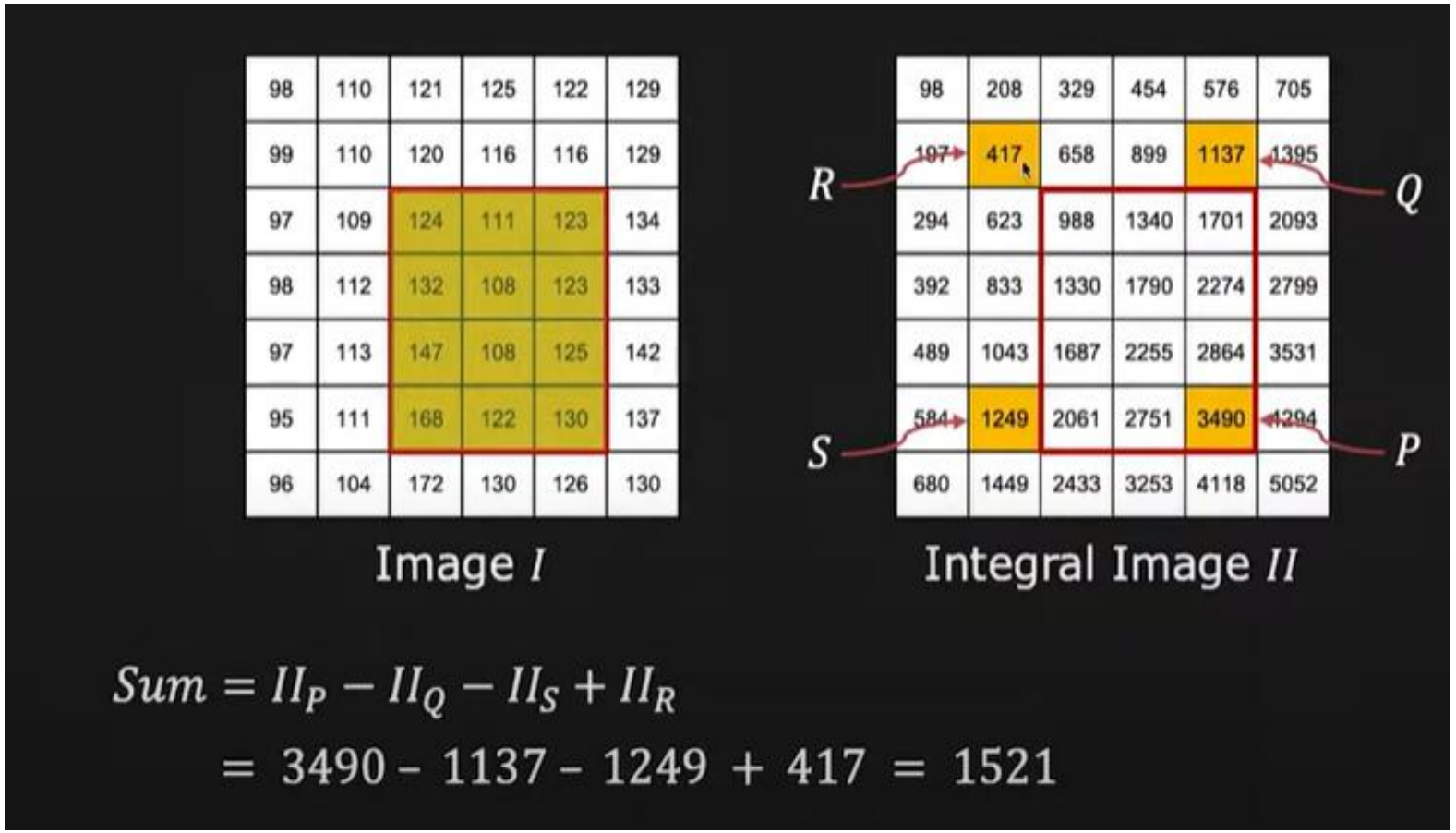
1	5	2
2	4	1
2	1	1

Integral Image:

1	6	8
3	12	15
5	15	19

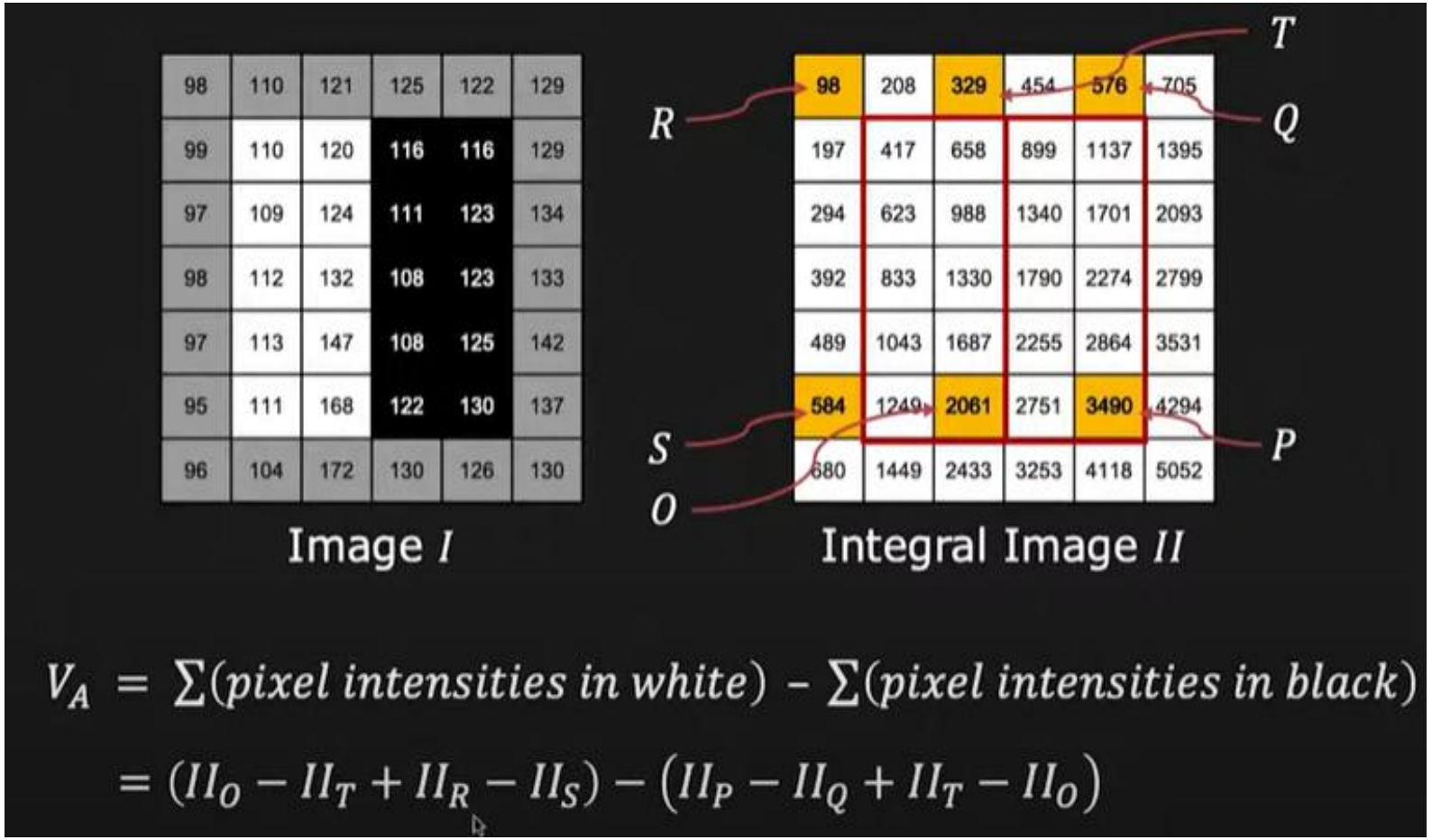
Integral Image

Integral Image



Computation cost only three addition

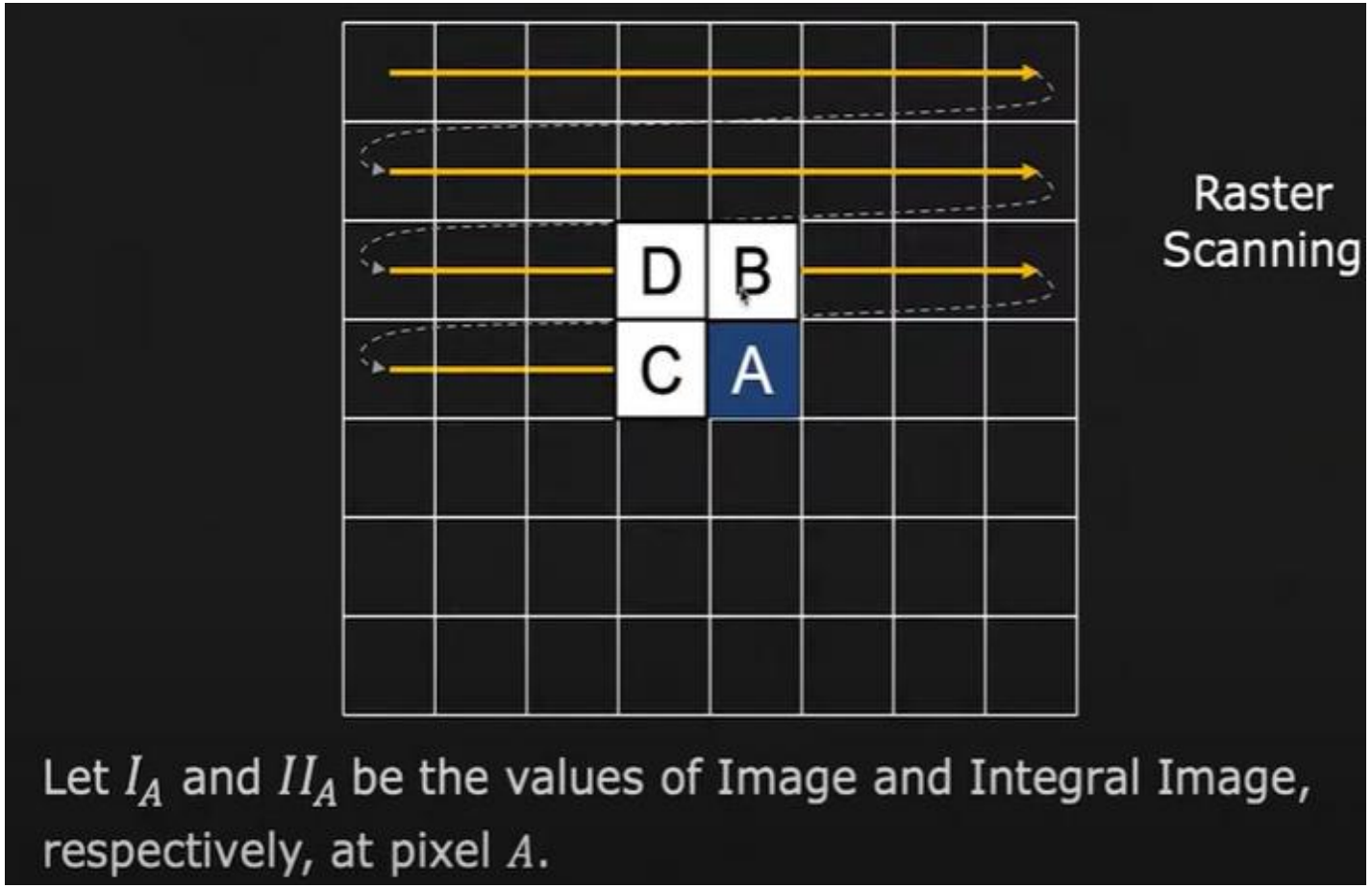
Haar response using integral image



$= (2061 - 329 + 98 - 584) - (3490 - 576 + 329 - 2061) = 64$

Computational Cost: Only 7 additions

Computing integral image



How face detector work

To select the specific Haar features to use, and to set threshold levels, Viola and Jones use a machine-learning method called AdaBoost.

AdaBoost combines many "weak" classifiers to create one "strong" classifier. "Weak" here means the classifier only gets the right answer a little more often than random guessing would. That's not very good. But if you had a whole lot of these weak classifiers, and each one "pushed" the final answer a little bit in the right direction, you'd have a strong, combined force for arriving at the correct solution.

AdaBoost selects a set of weak classifiers to combine and assigns a weight to each. This weighted combination is the strong classifier.

OpenCV Function

CascadeClassifier::detectMultiScale

(const Mat& **image**, vector<Rect>& **objects**,
double **scaleFactor**=1.1, int**minNeighbors**=3,
int **flags**=0, Size **minSize**=Size(), Size **maxSize**=Size())

OpenCV Function

cascade – Haar classifier cascade. It can be loaded from XML or YAML file usingLoad(). When the cascade is not needed anymore, release it usingcvReleaseHaarClassifierCascade(&cascade).

image – Matrix of the type CV_8U containing an image where objects are detected.

objects – Vector of rectangles where each rectangle contains the detected object.

scaleFactor – Parameter specifying how much the image size is reduced at each image scale.

minNeighbors – Parameter specifying how many neighbors each candidate rectangle should have to retain it.

flags – Parameter with the same meaning for an old cascade as in the function cvHaarDetectObjects. It is not used for a new cascade.

minSize – Minimum possible object size. Objects smaller than that are ignored.

maxSize – Maximum possible object size. Objects larger than that are ignored.

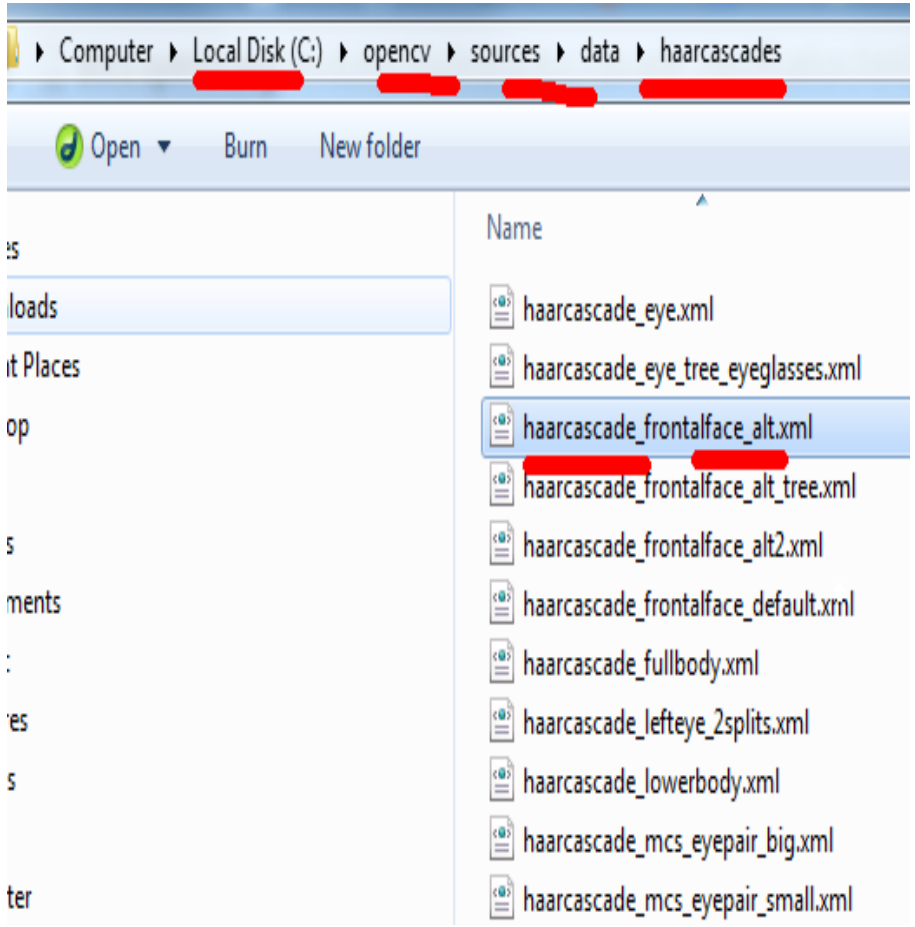
Code

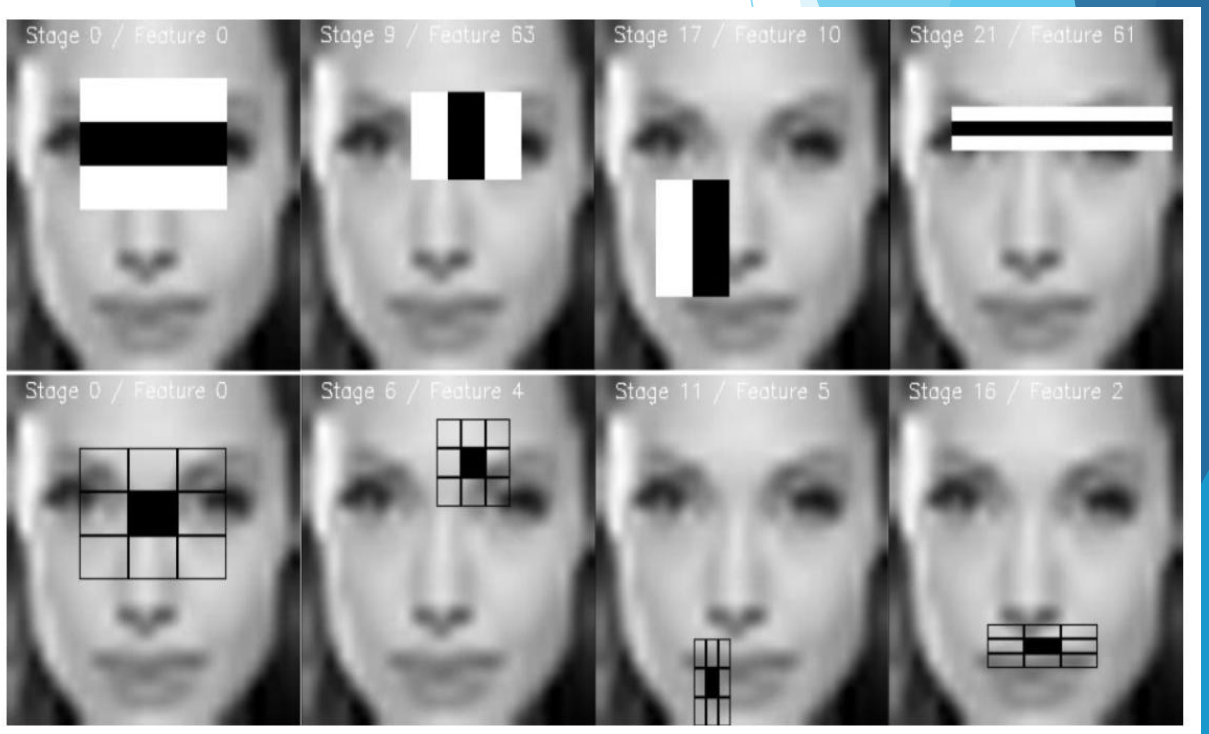
```
#include<opencv2\imgproc\imgproc.hpp>

using namespace cv;

String face_cascade = "C:\\opencv\\sources\\data\\haarcascades\\haarcascade_frontalface_alt.xml";
String eye_casecade = "C:\\opencv\\sources\\data\\haarcascades\\haarcascade_eye.xml";
CascadeClassifier face;
CascadeClassifier eye;

void FaceAndEyeDetect(Mat);
int main()
{
    VideoCapture v(0);
    assert(v.isOpened());
    bool b1 = face.load(face_cascade);
    bool b2 = eye.load(eye_casecade);
    assert(b1 && b2);

    Mat frame;
    while(true)
    {
        v.read(frame);
        FaceAndEyeDetect(frame);
        if((char)waitKey(20)==27) break;
    }
    return 0;
}
```

Code

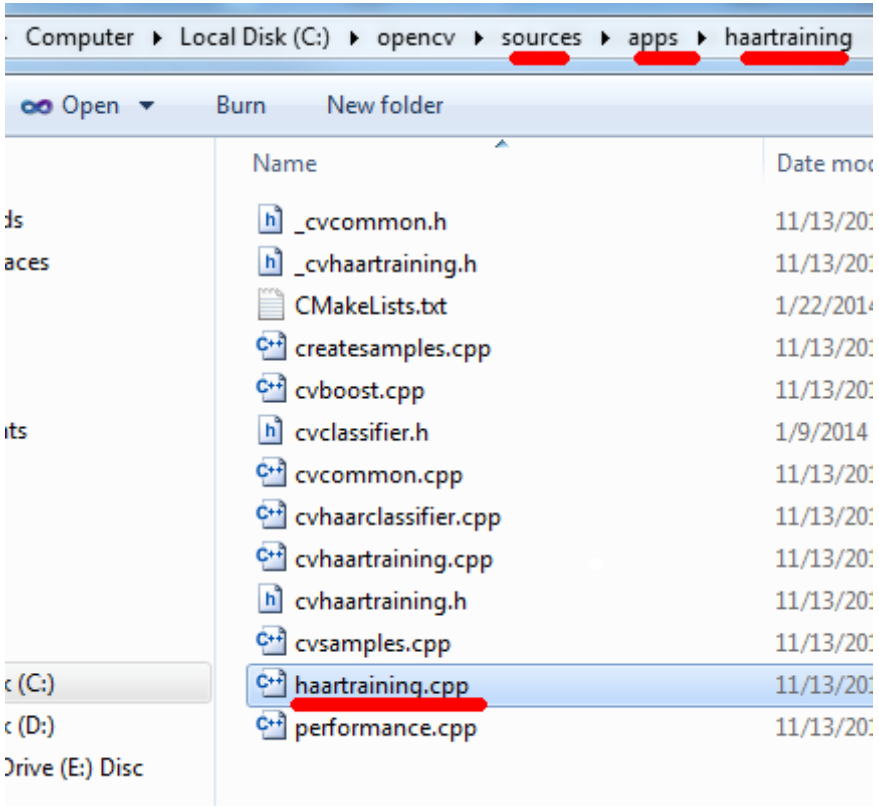
```
void FaceAndEyeDetect(Mat img)
{
    Mat gray;
    cvtColor(img,gray,CV_BGR2GRAY);

    vector<Rect> face_pos;
    face.detectMultiScale(gray,face_pos,1.1,2,0|CV_HAAR_SCALE_IMAGE,Size(10,10));

    for(int i=0;i<face_pos.size();i++)
    {rectangle(img,face_pos[i],Scalar(255,0,0),12);}
    //eye

    for(int i=0;i<face_pos.size();i++)
    {
        vector<Rect> eye_pos;
        Mat roi =gray(face_pos[i]);
        eye.detectMultiScale(roi,eye_pos,1.1,2,0|CV_HAAR_SCALE_IMAGE,Size(10,10));
        for(int j=0;j<eye_pos.size();j++)
        {
            Point center(face_pos[i].x+eye_pos[j].x+eye_pos[j].width*0.5,face_pos[i].y+eye_pos[j].y+eye_pos[j].height*0.5);
            int radious = cvRound((eye_pos[j].width+eye_pos[j].height)*0.25);
            circle(img,center,radious,Scalar(0,0,255),15,8,0);
        }

    }
    namedWindow("Search Face",0);
    resizeWindow("Search Face",300,300);
    imshow("Search Face",img);
}
```

Haarcascade_frontalface





Haartraining



https://docs.opencv.org/3.4/dc/d88/tutorial_traincascade.html

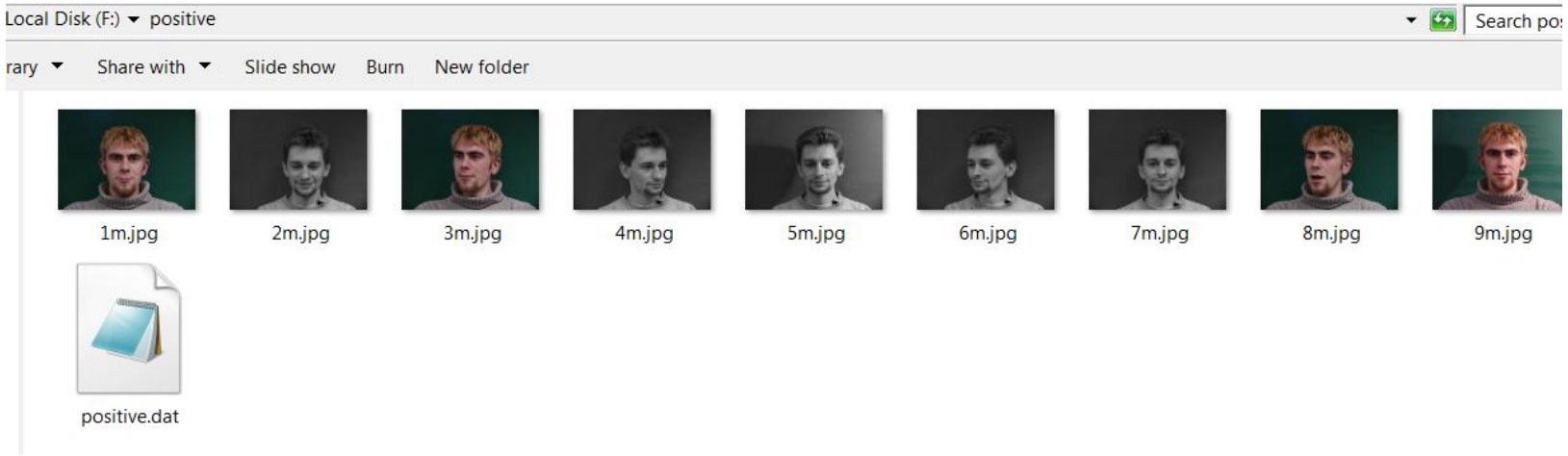
Command

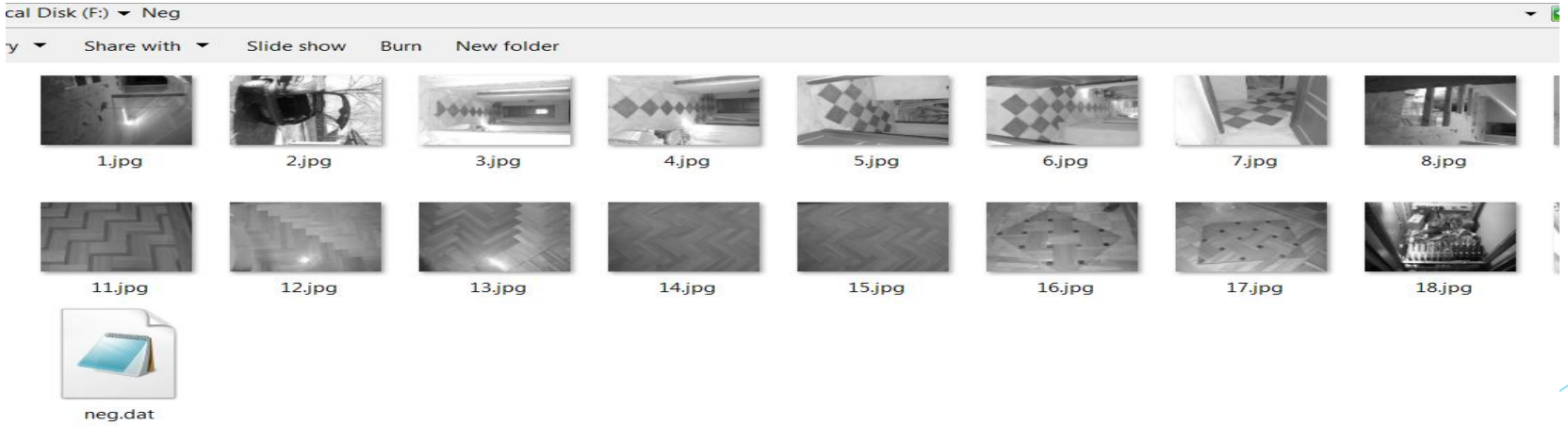
Normally DIR /b will return just the filename

```
F:\>cd positive

F:\positive>dir/b>positive.dat
```

```
F:\>cd Neg

F:\Neg>dir/b>neg.dat
```

Directory structure





Haar Training

“data” is the directory in which to store the output

“vec” is the .vec file containing the positive images

“bg” is a text file with a collection of paths to background (negative) images

“nstages” is the number of stages of boosting

“nsplits” is the number of split nodes in the decision trees which are the weak classifier for boosting

“minhitrate” and “maxfalsealarm” are cutoff values for hit rate and false alarm, per stage

HaarTraining steps

“npos” and “nneg” are the number of positive and negative images to be used from the given sets

“w” and “h” are the width and the height of the sample

“mem” is the amount of memory that should be used for precalculation. The default is 200MB

“mode” is either “BASIC” or “ALL”, stating whether (ALL) the full set of Haar features should be used (both upright and 45 degree rotated) or (BASIC) only upright features

command

- F:\>opencv_createsamples.exe -info “f:\Negative\neg.dat" -vec f:\Negative\neg.vec –num 20 -w 20 -h 20

- F:\>opencv_haartraining.exe -data “F:\Poistive\data\cascade" -vec “F:\prog\pos.vec" -bg “F:\Negative\neg.dat" -nstages14 –npos 20 -npos –nneg20 -mem 512 -mode All -w 20 -h 20

Use your own casecade classifier

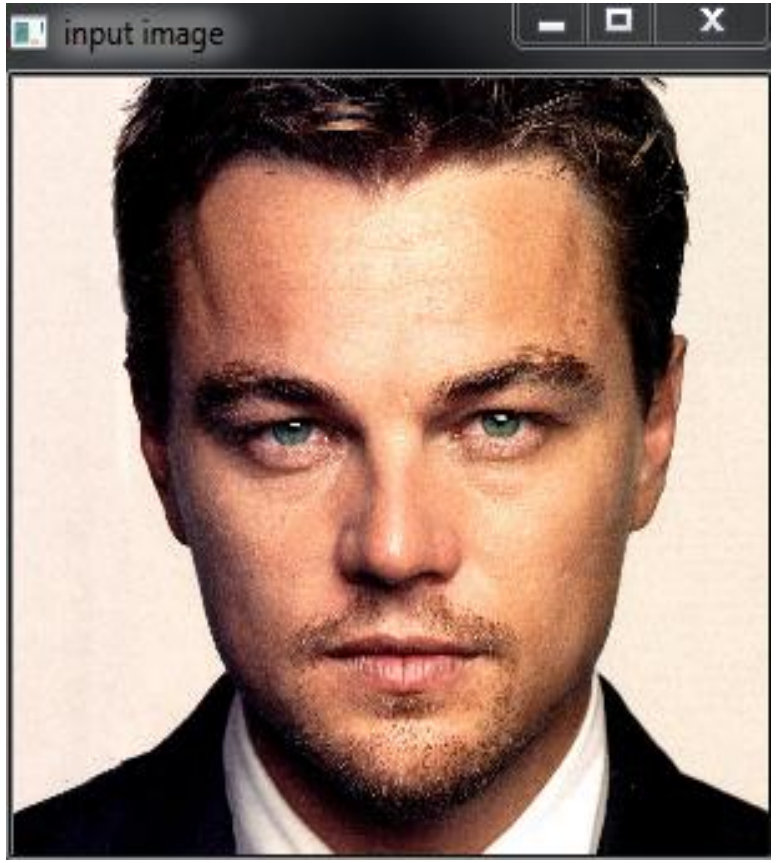
- The software that perform the viola-jones algorithm and creates the cascade file

- haarcascade_frontalface_alt.xml

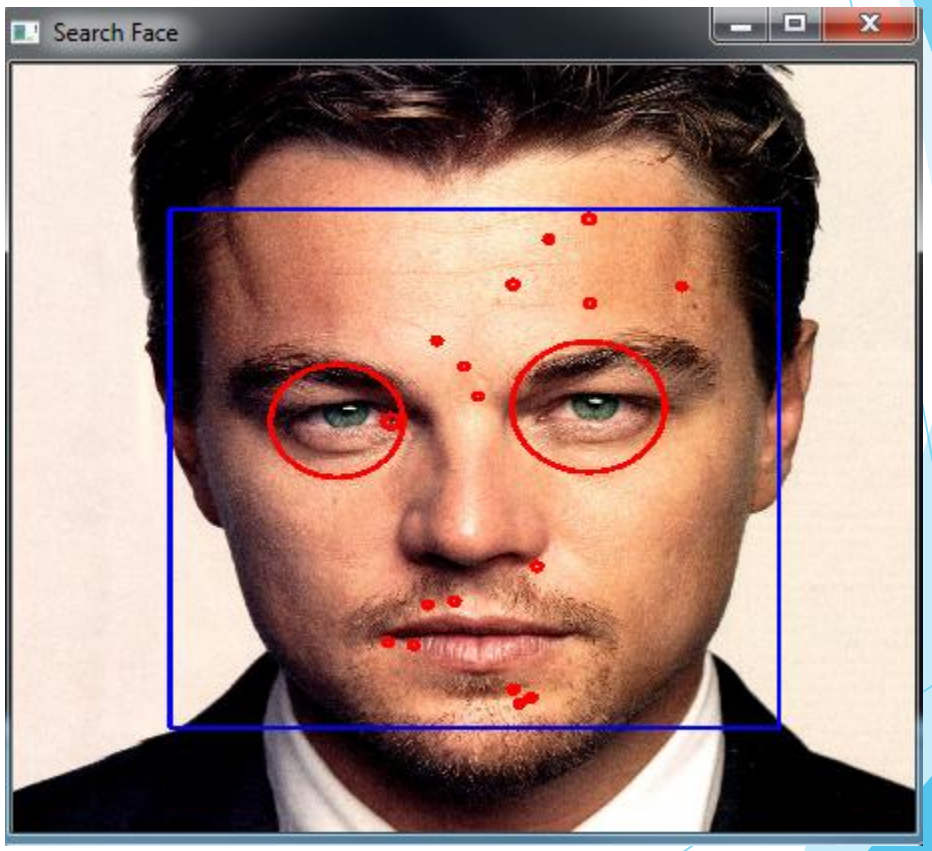
Haarcascade_frontalface_alt.xml

```
<opencv_storage>
<haarcascade_frontalface_alt type_id="opencv-haar-classifier">
  <size>20 20</size>
  <stages>
    <_>
      <!-- stage 0 -->
      <trees>
        <_>
          <!-- tree 0 -->
          <_>
            <!-- root node -->
            <feature>
              <rects>
                <_>3 7 14 4 -1.</_>
                <_>3 9 14 2 2.</_></rects>
              <tilted>0</tilted></feature>
            <threshold>4.0141958743333817e-003</threshold>
            <left_val>0.03379419073462249</left_val>
            <right_val>0.8378106951713562</right_val></_></_>
        <_>
          <!-- tree 1 -->
          <_>
            <!-- root node -->
            <feature>
              <rects>
                <_>1 2 18 4 -1.</_>
                <_>7 2 6 4 3.</_></rects>
              <tilted>0</tilted></feature>
            <threshold>0.01515133958309899</threshold>
            <left_val>0.1514132022857666</left_val>
            <right_val>0.7488812208175659</right_val></_></_>
        <_>
          <!-- tree 2 -->
          <_>
            <!-- root node -->
            <feature>
              <rects>
                < >1 7 15 9 -1.</ >
```

Output



Input Image



Output Image

Input image

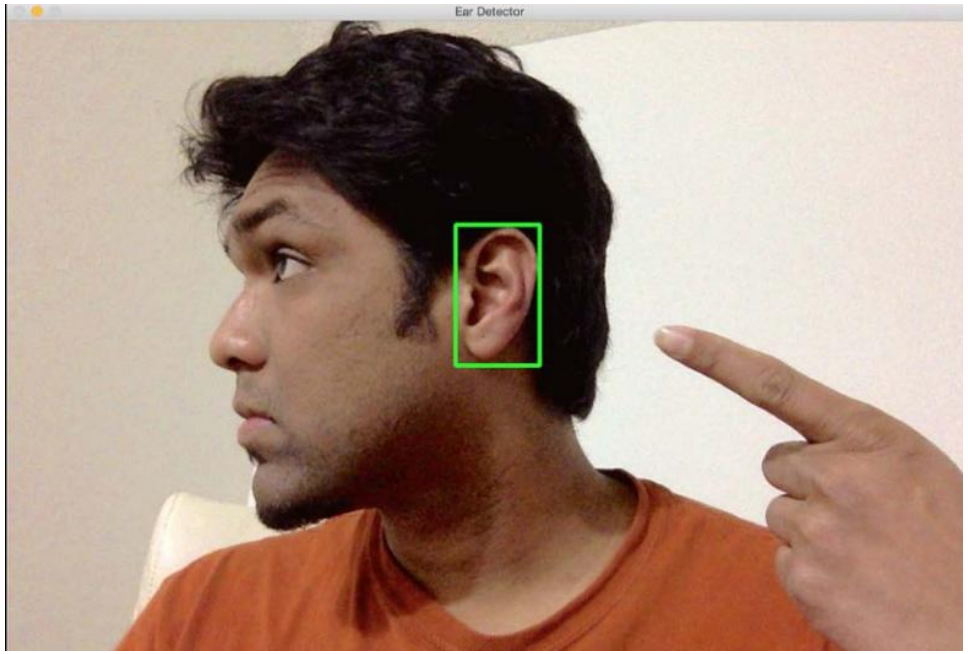
Fig. Input image

Output image

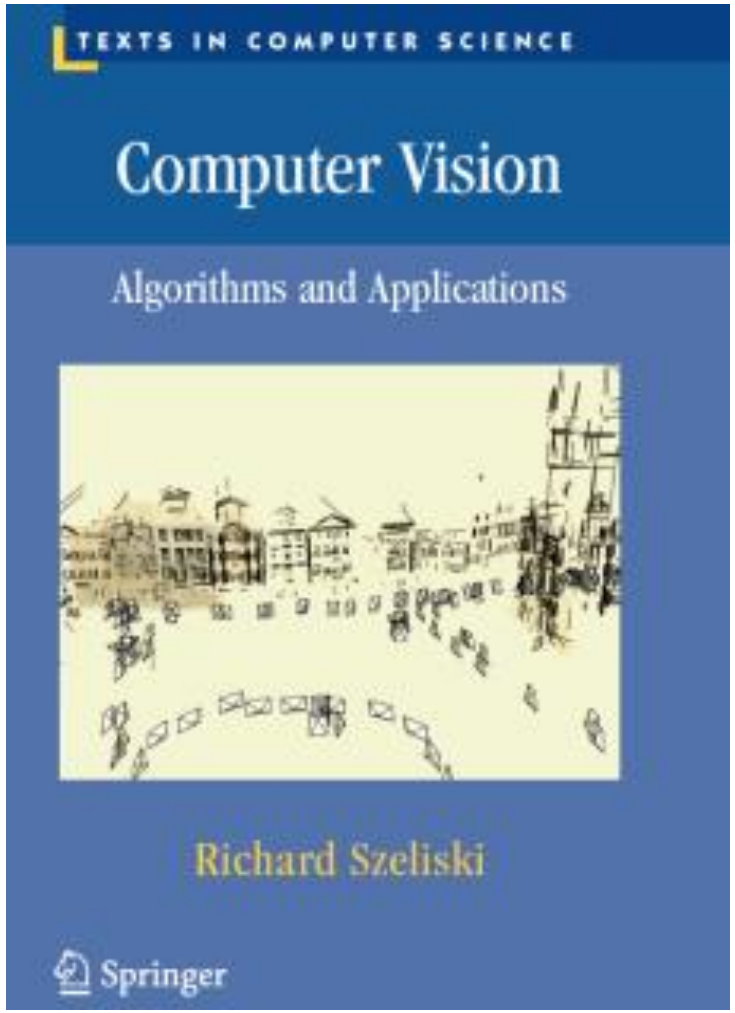
Fig. Output Image

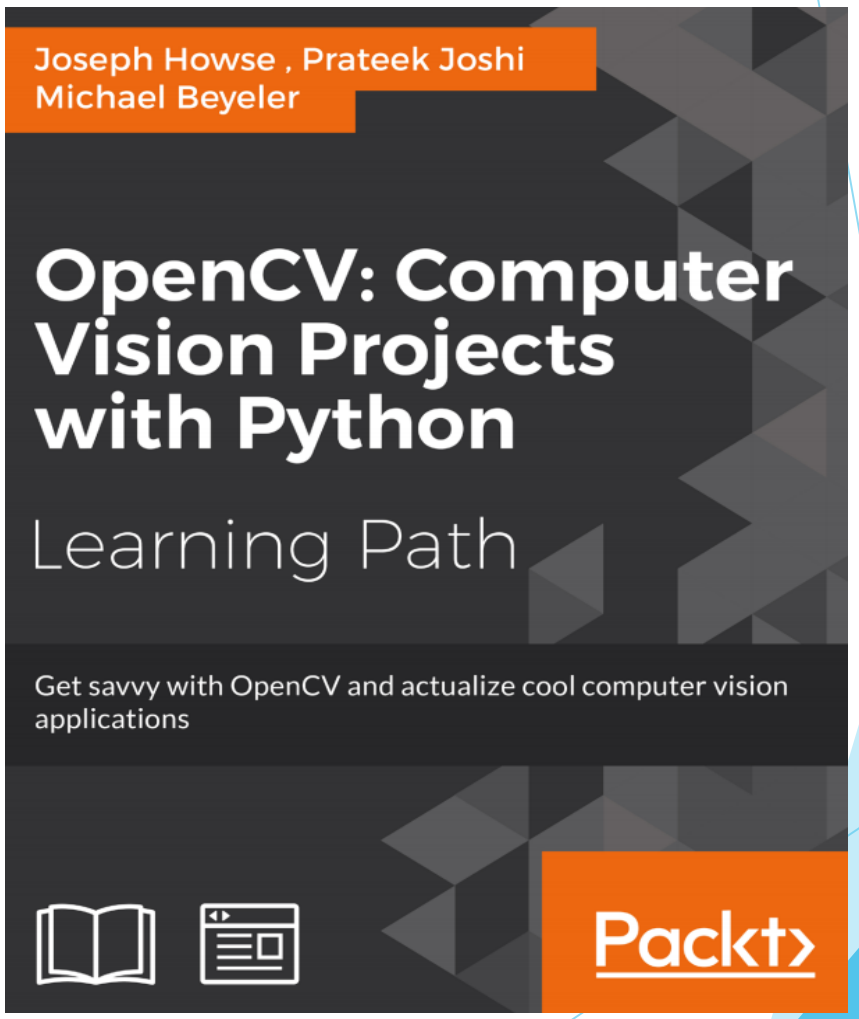
Exercise

- Use a Haarcascade classifier and detect Ear



Reference


TEXTS IN COMPUTER SCIENCE
Computer Vision
Algorithms and Applications
Richard Szeliski
Springer


Joseph Howse , Prateek Joshi
Michael Beyeler
OpenCV: Computer Vision Projects with Python
Learning Path
Get savvy with OpenCV and actualize cool computer vision applications
Packt

Reference

- https://docs.opencv.org/2.4/modules/objdetect/doc/cascade_classification.html
- https://en.wikipedia.org/wiki/Receiver_operating_characteristic

Question

Thank you